LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 4
Student Name: A.Rohit
Roll No.: A24126552067
Date: 31-08-2026

---

1. TITLE

Development of a Simple To-Do List using JavaScript ES6 Inheritance

---

2. AIM

To build an interactive To-Do List web application utilizing JavaScript ES6 class inheritance, object instantiation, and dynamic DOM manipulation for adding, completing, and deleting tasks.

---

3. OBJECTIVE

The objectives of this experiment are:

- To utilize ES6 `extends` and `super()` keywords for class inheritance.
- To implement methods within classes to generate dynamic HTML markup.
- To handle interactive events and manipulate specific DOM elements by ID dynamically.
- To implement state-driven UI updates based on container contents (e.g., checking if empty).

---

4. THEORY

Class inheritance in ES6 allows a class (Child) to inherit properties and methods from another class (Parent) using the `extends` keyword. The `super()` method is called in the child's constructor to execute the parent's constructor.

Dynamic DOM interactions involve adding new nodes, modifying styles inline (e.g., strikethrough for completed tasks), and removing nodes entirely from the document flow using the `remove()` method.

---

5. ALGORITHM / PROCEDURE

1. Define a base `Task` class with generic ID and text properties.
2. Define a `WebTask` class that extends `Task`, adding a method to return HTML markup.
3. Create an HTML input layout and an empty container for the task list.
4. Maintain a global counter to assign unique IDs to each new task instance.
5. On 'Add Task', instantiate `WebTask`, append its HTML to the list, and update the counter.
6. Implement a 'Complete' function that finds the task text by ID and applies text-decoration styling.
7. Implement a 'Delete' function that completely removes the specific task element using `remove()`.
8. Implement a helper function to toggle an empty state message if the list is empty.

---

6. PROGRAM

Source File: task4.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Simplest To-Do List</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            padding: 20px;
            background: #f5f5f5;
        }
        .container {
            max-width: 450px;
            margin: auto;
            background: white;
            padding: 25px;
            border-radius: 10px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
        }
        input {
            width: 70%;
            padding: 10px;
            border: 1px solid #ddd;
            border-radius: 5px;
        }
        button {
            padding: 10px 15px;
            background: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
        .task-item {
            background: #f9f9f9;
            padding: 10px;
            margin-top: 10px;
            border-left: 4px solid #007bff;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }
    </style>
</head>
<body>

<div class="container">
    <h2>My To-Do List</h2>

    <div>
        <input type="text" id="taskInput" placeholder="Enter a task.............">
        <button onclick="addTask()">Add Task</button>
    </div>
```

```
    <p id="emptyMsg">No tasks available.</p>

    <!-- The list container -->
    <div id="taskList"></div>
</div>

<script>
    // 1. THE PARENT CLASS (Basic properties)
    class Task {
        constructor(id, text) {
            this.id = id;
            this.text = text;
        }
    }

    // 2. THE CHILD CLASS (Inherits properties, adds a method for HTML)
    class WebTask extends Task {
        constructor(id, text) {
            super(id, text);
        }

        getHTML() {
            // Notice we give the div and span special IDs based on their number
            return `
                <div class="task-item" id="taskBox-${this.id}">
                    <span id="taskText-${this.id}">${this.text}</span>
                    <div>
                        <button style="background: #28a745;"
onclick="completeTask(${this.id})">Complete</button>
                        <button style="background: #dc3545;"
onclick="deleteTask(${this.id})">Delete</button>
                    </div>
                </div>
            `;
        }
    }

    // 3. MAIN LOGIC (No Arrays!)
    let taskCounter = 0; // Just a simple number to keep track of tasks

    function addTask() {
        let inputBox = document.getElementById("taskInput");
        let text = inputBox.value.trim();

        if (text === "") {
            alert("Please enter a task.");
            return;
        }

        // Create the task object
        let newTask = new WebTask(taskCounter, text);

        // Add the HTML string directly to the bottom of the list using +=
        document.getElementById("taskList").innerHTML += newTask.getHTML();
```

```
        taskCounter++; // Increase the number for the next task
        inputBox.value = ""; // Clear the text box

        checkEmptyMessage(); // Hide the empty message
    }

    function completeTask(id) {
        // Just find the specific text span by its ID and strike it out
        let specificText = document.getElementById("taskText-" + id);
        specificText.style.textDecoration = "line-through";
        specificText.style.color = "gray";
    }

    function deleteTask(id) {
        // Find the specific task box by its ID and completely remove it from the page
        let specificBox = document.getElementById("taskBox-" + id);
        specificBox.remove();

        checkEmptyMessage(); // Check if we need to show the empty message again
    }

    // A helper function to show or hide the "No tasks" message
    function checkEmptyMessage() {
        let listDiv = document.getElementById("taskList");
        let emptyMsg = document.getElementById("emptyMsg");

        // If the HTML inside the list is completely empty, show the message
        if (listDiv.innerHTML.trim() === "") {
            emptyMsg.style.display = "block";
        } else {
            emptyMsg.style.display = "none";
        }
    }
</script>

</body>
</html>
```

---

7. CODE EXPLANATION

Code / Syntax	Explanation
class WebTask extends Task	Defines a child class inheriting from the Parent Task class.
super(id, text)	Calls the constructor of the Parent class to initialize shared properties.
element.innerHTML += ...	Appends new HTML content to the existing content of a container.
element.style.textDecoration	Dynamically modifies inline CSS to apply strikethrough styles.

element.remove()	Removes the specific HTML element from the DOM entirely.

---

8. TEST CASES AND EXECUTION DATA

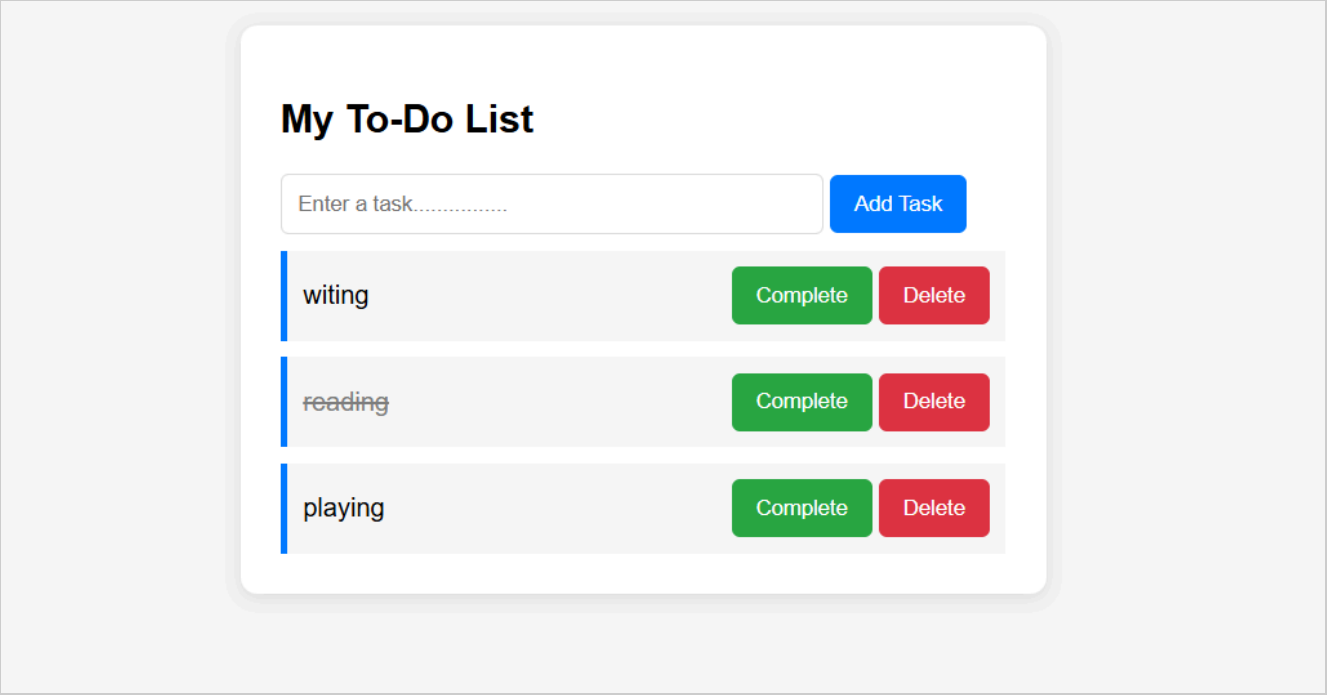
The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Add Task	Input text and click Add	Task renders dynamically in the list	Pass
TC-02	Complete Task	Click Complete on a task	Text is crossed out (strikethrough)	Pass
TC-03	Delete Task	Click Delete on a task	Task is entirely removed from the page	Pass
TC-04	Empty State	Delete all tasks	'No tasks available' message reappears	Pass

---

9. OUTPUT

Output 1



---

10. RESULT

Thus, the To-Do list application utilizing JavaScript ES6 class inheritance and advanced DOM manipulation was successfully implemented and verified.